

재현 가이드 — 문의 접수 분류 자동화 (Make vs n8n)

이 문서만 보고 처음부터 끝까지 동일한 워크플로우를 다시 만들 수 있도록 작성했다. 동료 평가자가 검증할 때, 또는 시간이 지나 본인이 다시 구현할 때 사용한다.

소요 시간 기준: 준비 20분 + Make 60분 + n8n 90분 ≒ 약 3시간

0. 한눈에 보기

만들 것

```
Trigger(Webhook) → 조건 분기(priority == "urgent") ⌋ True → Sheets(긴급문의) → Discord(긴급알림)  
⌋ False → Sheets(일반문의) → Discord(일반알림)
```

동일한 이 구조를 **Make**와 **n8n** 두 도구로 각각 만든다.

준비물 체크리스트

- ☐ Google 계정 (Sheets 생성 + GCP 콘솔 접근 가능해야 함)
- ☐ Discord 계정
- ☐ Make 계정 (무료)
- ☐ Docker Desktop 설치 가능한 PC (n8n 셀프호스팅용)
- ☐ PowerShell (Windows 기준)

전체 순서

단계	내용	예상 시간
1	공통 준비물 생성 (Sheets, Discord)	20분
2	Make 계정 생성 및 플랜 확인	10분
3	Make 워크플로우 구현	50분
4	Make 분기 테스트 6건	25분
5	Docker + n8n 환경 구축	15분
6	n8n Google 인증 구성	20분
7	n8n 워크플로우 구현	50분
8	n8n 분기 테스트 6건	20분

⌚ 시간을 측정하며 진행할 것. 비교 보고서의 "설정 난이도" 항목은 실측값이 근거가 된다. 측정 구간은 ① 초기 진입 ② 워크플로우 구축 ③ 분기 테스트 세 개로 나눈다. 공통 준비물 생성(1단계)은 두 도구가 공유하므로 측정에서 제외한다.

1. 공통 준비물 만들기

두 도구가 같은 시트, 같은 Discord 채널을 사용한다. 도구별로 따로 만들면 "동일 워크플로우" 비교 전제가 무너진다.

1-1. Google Sheets

새 스프레드시트를 만들고 이름을 **문의접수-자동화** 로 지정한다.

시트 탭을 2개 만든다. 이름은 정확히 아래와 같이 한다. 나중에 도구에서 이름으로 참조하므로 오타가 있으면 연결이 끊긴다.

- 긴급문의
- 일반문의

양쪽 시트 1행에 동일한 헤더 7개를 입력한다.

A	B	C	D	E	F	G
수신시각	ticket_id	customer	category	priority	message	source

source 컬럼이 중요하다. 두 도구가 같은 시트에 쓰기 때문에, 어느 도구가 기록했는지 이 컬럼으로 구분한다.

1-2. Discord

서버가 없으면 새로 만든다. **서버 이름에 실명을 넣지 않는다.** (캡처에 그대로 노출된다)

텍스트 채널 2개를 만든다. 음성 채널은 Webhook을 붙일 수 없다.

- 긴급알림
- 일반알림

각 채널마다 Webhook을 발급한다.

채널 우클릭 → 채널 편집 → 연동 → 웹후크 → 새 웹후크 → 웹후크 URL 복사

발급받은 URL 2개를 메모장 등에 임시 보관한다.

🔒 이 URL은 비밀정보다. 이것만 알면 누구나 해당 채널에 메시지를 보낼 수 있다. 문서·캡처에 노출되지 않게 하고, 작업이 끝나면 재발급한다.

1-3. 테스트 데이터 설계

분기 양쪽을 모두 태우고 경계 조건까지 확인하도록 6건을 준비한다. **Make**는 T-1001번대, **n8n**은 T-2001번대를 써서 시트에서 구분되게 한다.

#	ticket_id	priority	기대 경로	검증 의도
1	T-x001	urgent	True	정상 긴급
2	T-x002	urgent	True	정상 긴급
3	T-x003	urgent	True	정상 긴급
4	T-x004	normal	False	정상 일반
5	T-x005	low	False	normal 이 아닌 값도 걸러지는가
6	T-x006	필드 없음	False	필드 자체가 없을 때의 처리

5-6번을 반드시 포함한다. 특히 6번은 두 도구의 조건 분기 구현 방식 차이가 드러나는 지점이다.

2. Make 구현

2-1. 계정 생성 및 플랜 확인 🕒 측정 시작 (초기 진입)

1. Make 가입 후 로그인
2. 좌측 **My Plan → Subscription** 이동
3. 아래 3가지를 확인하고 **캡처**한다
 - 월 무료 실행량 (Free · 1,000 credits/month)
 - 활성 시나리오 수 제한 (2개)
 - 시나리오 실행시간 제한 (5분)

📷 **make-00-plan.png** 이 캡처가 보고서의 "무료 플랜 범위" 항목 근거가 된다.

⚠️ 플랜 카드에 **조건 분기(Router)** 포함 여부는 명시되어 있지 않다. 실제로 배치해 봐야 확인된다. 유료 전환 프로모션 배너가 뜨더라도 **사용하지 않는다**.

🕒 측정 종료 (초기 진입)

2-2. 시나리오 생성 🕒 측정 시작 (워크플로우 구축)

1. **Scenarios → Create a new scenario**

- 좌측 상단 **New scenario** 글자를 클릭해 이름을 **P1-문의접수분류-Make** 로 변경
- 좌측 **Build with Maia** 패널을 **닫는다** (X 버튼)

🚫 **AI 빌더(Maia)는 사용하지 않는다.** 모듈 구조를 직접 이해하는 것이 목적이다. 다만 "AI 빌더 제공 여부"는 n8n과의 비교 항목으로 기록해 둘 만하다.

2-3. Trigger — Webhook 배치

- 가운데 보라색 **+** 클릭
- Webhooks** 검색 → **Custom webhook** 선택
- Add** → 이름 **문의접수** → 저장
- Webhook URL 복사**

2-4. 데이터 구조 인식 ⚠️ 첫 번째 함정

Make는 **실제 요청을 한 번 받아야** 필드를 인식한다.

- 모듈 설정창에서 **Detect new values** 클릭 → **Waiting for data...** 대기 상태 진입
- 그 상태를 유지한 채로** PowerShell에서 아래 실행

```
$body='{ "ticket_id": "T-1001", "customer": "고객A", "category": "결제", "priority": "urgent", "message": "결제가 두 번 청구되었습니다"}'; Invoke-RestMethod -Uri "복사한WebhookURL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

- Accepted** 응답이 오고, Make 화면이 **5 values detected and ready to map** 으로 바뀌면 성공
- Save**

📷 **make-01-webhook-detected.png** — 필드 5개가 인식된 화면

여기서 자주 막히는 3가지

증상	원인	해결
Expected ',' or '}' in JSON	JSON 닫는 중괄호 } 누락	JSON 문법 재확인
There is no scenario listening for this webhook	Make가 대기 상태가 아님	Detect new values 를 먼저 누른다
'>>' 용어가 인식되지 않습니다	PowerShell이 여러 줄 입력 상태	Ctrl+C 로 초기화 후 한 줄로 붙여 넣기

💡 한글이 깨지면 **charset=utf-8** 과 **UTF8.GetBytes** 부분이 빠졌는지 확인한다.

2-5. 조건 분기 — Router 배치

1. Webhook 모듈 우측 반원 + → Router 검색 → 배치
2. 자동으로 두 갈래(1st / 2nd) 경로가 생성된다

✅ Router가 자물쇠 없이 배치되면 Free 플랜에서 조건 분기 사용이 가능하다는 뜻이다. 이 사실 자체가 S 무료 플랜 범위의 근거가 된다.

2-6. Action 1 — Google Sheets (경로 A)

1. 1st(위쪽) 경로의 + → Google Sheets → Add a Row
2. Connection → Add → 구글 계정 선택 → 승인 (1~2분)
3. Click here to choose file → 문의접수-자동화 선택
4. Sheet Name → 긴급문의
5. Values 영역에 A~G 매핑

칸	값
A	{{now}}
B	ticket_id (변수)
C	customer (변수)
D	category (변수)
E	priority (변수)
F	message (변수)
G	Make (직접 입력)

⚠️ 두 번째 함정 — 변수 매핑 무음 실패 {{ticket_id}} 같은 문자열을 손으로 치거나 붙여넣으면 안 된다. 겉보기에는 검은 변수 태그로 바뀌어 정상처럼 보이지만, 실제로는 모듈 참조가 없어 오류 없이 값만 비어 있는 상태로 실행된다. 반드시 우측 변수 패널에서 필드를 클릭해 삽입한다.

Make의 Sheets 매핑은 헤더 이름이 아니라 A~Z 위치 기반이다. 한 칸만 밀려도 데이터가 어긋난다. H~Z 칸은 비워 두면 된다.

2-7. Action 2 — Discord 알림 (경로 A)

1. Sheets 모듈 우측 + → HTTP → Make a request (인증 없는 기본형)
2. 설정

항목	값
Authentication type	None 계열

항목	값
URL	긴급알림 Webhook URL
Method	POST
Body content type	application/json
Body input method	JSON string
Parse response	No

3. Body content에 입력 (변수는 우측 패널에서 삽입)

```
{"content": "🔴 긴급 문의 접수\nID: {{ticket_id}}\n고객: {{customer}}\n분류: {{category}}\n내용: {{message}}"}

```

💡 Parse response 를 No 로 두는 이유: Discord Webhook은 성공 시 빈 응답(204)을 반환하는데, Yes 면 Make가 빈 응답을 JSON으로 파싱하려다 실패할 수 있다.

2-8. Action 3-4 — 경로 B

2nd(아래쪽) 경로에 같은 구성을 만든다. 모듈 우클릭 → Clone 으로 복제하면 빠르다.

변경할 것	값
Sheets → Sheet Name	일반문의
HTTP → URL	일반알림 Webhook URL
HTTP → 문구	📧 일반 문의 접수

2-9. Router 필터 설정

경로 A (1st) — Router에서 1st로 나가는 선을 클릭 → Set up a filter

항목	값
Label	긴급
조건	priority (변수) — Text operators: Equal to — urgent (직접 입력)

경로 B (2nd) — 같은 방식으로 필터 설정

항목	값
Label	일반(fallback)

항목	값
조건	priority (변수) — Text operators: Not equal to — urgent

⚠ 세 번째 함정 — fallback route를 찾지 못하는 문제 원래는 경로 B를 "fallback route"로 지정하면 되지만, UI에서 해당 옵션을 찾기 어렵다. Not equal to 필터로 대체하면 동작이 동일하다. 이 우회 경험 자체가 "조건 분기 구현 방식" 비교 항목의 근거가 되므로 기록해 둔다.

조건값 urgent 는 직접 타이핑한다. 왼쪽은 변수, 오른쪽은 고정값이다.

2-10. 저장 및 구조 캡처

Ctrl+S 또는 하단 저장 아이콘으로 저장한다. URL이 /scenarios/숫자/edit 로 바뀌면 저장된 것이다.

📸 make-02-workflow-full.png — 모듈 6개와 분기 라벨이 모두 보이는 캔버스 전체 캡처 전에 Maia 패널을 닫고, 화면 알림을 끈다.

🕒 측정 종료 (워크플로우 구축) — 첫 성공 실행 시점까지

2-11. 분기 테스트 6건 🕒 측정 시작 (분기 테스트)

각 요청마다 Run once 를 먼저 누른 뒤 명령을 실행한다.

1번 (긴급)

```
$body='{ "ticket_id": "T-1001", "customer": "고객A", "category": "결제", "priority": "urgent", "message": "결제가 두  
번 청구되었습니다"}'; Invoke-RestMethod -Uri "URL" -Method Post -ContentType "application/json; charset=utf-  
f-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

2번 (긴급)

```
$body='{ "ticket_id": "T-1002", "customer": "고객B", "category": "배송", "priority": "urgent", "message": "주문이 취  
소되지 않습니다"}'; Invoke-RestMethod -Uri "URL" -Method Post -ContentType "application/json; charset=utf-  
8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

3번 (긴급)

```
$body='{ "ticket_id": "T-1003", "customer": "고객C", "category": "결제", "priority": "urgent", "message": "카드 결제  
가 실패합니다"}'; Invoke-RestMethod -Uri "URL" -Method Post -ContentType "application/json; charset=utf-  
8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

4번 (normal)

```
$body='{ "ticket_id": "T-1004", "customer": "고객D", "category": "환불", "priority": "normal", "message": "환불 절차  
를 문의합니다"}'; Invoke-RestMethod -Uri "URL" -Method Post -ContentType "application/json; charset=utf-  
8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

5번 (low)

```
$body='{ "ticket_id": "T-1005", "customer": "고객E", "category": "기타", "priority": "low", "message": "영업시간이 공  
금합니다"}'; Invoke-RestMethod -Uri "URL" -Method Post -ContentType "application/json; charset=utf-8" -B  
ody ([System.Text.Encoding]::UTF8.GetBytes($body))
```

6번 (priority 필드 없음)

```
$body='{ "ticket_id": "T-1006", "customer": "고객F", "category": "기타", "message": "필드 누락 테스트입니다"}'; Invok  
e-RestMethod -Uri "URL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Tex  
t.Encoding]::UTF8.GetBytes($body))
```

⚠ **네 번째 함정 — Webhook 큐 적재** Make는 대기 상태가 아닐 때 들어온 요청을 버리지 않고 큐에 쌓아둔다. 나중에 **Run once** 를 누르면 큐에서 하나씩 꺼내 처리하므로, **의도치 않은 중복 기록**이 생긴다. 시트에 같은 티켓이 여러 번 보이면 이 때문이다. 최종 캡처 전에 중복 행을 정리한다.

결과 확인 3곳

1. 캔버스 모듈에 초록 체크
2. Google Sheets에 행 추가 (컬럼이 밀리지 않았는지 확인)
3. Discord 채널에 메시지 (**값이 비어 있지 않은지 확인** — 2-6의 함정)

🕒 측정 종료 (분기 테스트)

2-12. Make 캡처 목록

시나리오 상세 화면 → **HISTORY** 탭에서 개별 실행을 열면 모듈별 실행 결과가 보인다.

파일명	내용	확인 포인트
<code>make-00-plan.png</code>	Free 플랜 한도	credits, 활성 시나리오 수
<code>make-01-webhook-detected.png</code>	Trigger 데이터 인식	<code>5 values detected</code>
<code>make-02-workflow-full.png</code>	전체 구조	모듈 6개 + 분기 라벨
<code>make-03-run-branch-a.png</code>	긴급 경로 실행	긴급 1건 / 일반 0건, 로그에 <code>did not pass through the filter</code>
<code>make-04-run-branch-b.png</code>	일반 경로 실행	위와 정확히 반대
<code>make-05-sheets-result.png</code>	시트 기록 결과	두 시트 분리 기록
<code>make-06-discord-urgent.png</code>	긴급 채널	채널명 + 값 채워진 메시지
<code>make-07-discord-normal.png</code>	일반 채널	채널명 + 값 채워진 메시지

3. n8n 구현

3-1. Docker + n8n 구동 🌀 측정 시작 (초기 진입)

1. Docker Desktop 설치

`docker.com` → Windows AMD64 버전 다운로드 → 설치 → 재부팅

2. WSL 설치 (설치 중 `WSL not installed` 오류가 나면)

관리자 권한 PowerShell에서 실행 후 **재부팅**한다.

```
wsl --install
```

3. Docker Desktop 실행

좌측 하단에 `Engine running` 이 표시되면 준비 완료다. 로그인은 하지 않아도 된다.

4. n8n 컨테이너 실행

```
docker run -it --rm -p 5678:5678 -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

`Editor is now accessible via: http://localhost:5678` 가 출력되면 성공이다.

🔒 이 PowerShell 창을 닫으면 n8n이 종료된다. 테스트 요청은 반드시 새 창에서 보낸다. 컨테이너가 꺼지면 워크플로우 전체가 정지한다. 셀프호스팅의 특성이다.

📸 n8n-01-docker-run.png — 구동 로그 (버전 번호와 접속 주소가 보이게)

5. 계정 생성

브라우저에서 localhost:5678 접속 → 계정 생성 화면

로컬 인스턴스이므로 실제 인증은 없다. 캡처에 실제 이메일이 남지 않도록 가상 주소를 쓴다.

항목	예시
Email	test@example.com
Password	Test1234 (8자 이상, 숫자·대문자 포함)

🔑 비밀번호는 브라우저 비밀번호 관리자나 개인 메모에 보관한다. 프로젝트 폴더 안에 저장하지 않는다.

6. 라이선스 안내 화면

Get paid features for free 팝업이 뜨면 Skip 을 누른다.

📸 n8n-01c-license-prompt.png 이 화면이 "셀프호스팅은 무제한이지만 일부 기능(고급 디버깅, 실행 검색·태깅, 폴더)은 라이선스 키가 필요하다"는 근거가 된다.

🕒 측정 종료 (초기 진입)

3-2. 워크플로우 생성 🕒 측정 시작 (워크플로우 구축)

Create Workflow → 이름을 P1-문의접수분류-n8n 으로 변경

3-3. Trigger — Webhook 노드

1. Add first step → On webhook call 선택
2. HTTP Method를 POST 로 변경 (기본값이 GET이다)
3. Path는 자동 생성값 그대로 사용
4. Test URL 탭의 주소를 복사

💡 n8n은 Test URL과 Production URL을 분리해서 제공한다. Make에는 없는 개념이다.

3-4. 데이터 구조 인식

1. Listen for test event 클릭 → 대기 상태 진입
2. 새 PowerShell 창에서 실행

```
$body='{ "ticket_id": "T-2001", "customer": "고객A", "category": "결제", "priority": "urgent", "message": "결제가 두  
번 청구되었습니다"}'; Invoke-RestMethod -Uri "복사한TestURL" -Method Post -ContentType "application/json; ch  
arset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

3. **Workflow was started** 응답이 오면 성공

 **n8n-02-webhook-node.png** — OUTPUT에 JSON이 표시된 화면

⚠ **다섯 번째 함정 — 데이터가 body에 중첩된다** n8n은 수신 데이터를 **headers** / **params** / **query** / **body**로 구조화한다. 실제 페이로드는 **body** 안에 있으므로 참조 경로가 **{{ \$json.body.priority }}**가 된다. **\$json.priority**로 쓰면 값이 잡히지 않는다. **좌측 INPUT 패널에서 드래그하면 경로가 자동 계산되므로** 드래그를 권장한다.

⚠ **URL 복사 시 UUID를 정확히 옮긴다.** 한 글자만 달라도 404가 나며, 원인을 찾기 어려워 시간을 크게 잃는다. 복사 버튼을 쓰고, 눈으로 대조한다.

3-5. 조건 분기 — IF 노드

1. Webhook 노드 우측 **+** → **If** 검색 → 추가
2. 조건 설정

항목	값	입력 방법
왼쪽 값	{{ \$json.body.priority }}	INPUT 패널에서 body → priority 드래그
연산자	String → is equal to	드롭다운
오른쪽 값	urgent	직접 타이핑

3. **Execute step** 을 눌러 단독 검증

OUTPUT에 **True Branch (1 item)** / **False Branch** 탭이 생기면 정상이다.

 **n8n-02b-if-node.png** — True/False 탭이 보이는 화면

✅ **Make와의 결정적 차이:** IF 노드는 **true / false** 출력이 **처음부터 두 개** 제공된다. Make에서 fallback route를 찾지 못해 우회했던 문제가 여기서는 발생하지 않는다.

✅ **노드 단위 실행이 가능하다.** 워크플로우 전체를 돌리지 않고 특정 노드만 검증할 수 있다.

3-6. Google Sheets 인증 🔄 별도 측정 권장 — 가장 오래 걸리는 구간

1. IF 노드의 **true 출력** **+** → **Google Sheets** → **Append row in sheet**
2. **Set up credential** → **OAuth2** 선택

3. 화면의 OAuth Redirect URL을 복사해 둔다

이제 Google Cloud 콘솔에서 OAuth 앱을 직접 만들어야 한다.

① 프로젝트 생성

`console.cloud.google.com` 접속 → 상단 프로젝트 선택 → 새 프로젝트 → 이름 `n8n-automation` → 만들기

💡 브라우저에 여러 구글 계정이 로그인되어 있으면 다른 계정으로 잡힐 수 있다. URL 뒤에 `?authuser=1` (숫자를 바꿔가며) 붙이거나, 크롬 프로필을 분리하면 해결된다. 스프레드시트를 만든 계정과 동일한 계정이어야 한다.

② API 활성화 (2개 모두)

API 및 서비스 → 라이브러리 에서 검색 후 각각 사용 클릭

- Google Sheets API
- Google Drive API ← 시트 목록을 불러오는 데 필요하다. 빠뜨리면 드롭다운이 빈다.

③ OAuth 동의 화면 구성

OAuth 동의 화면 (또는 Google 인증 플랫폼)

항목	값
User Type	외부
앱 이름	<code>n8n-local</code>
지원 이메일 / 개발자 연락처	본인 계정
범위(Scopes)	추가하지 않고 넘어감
테스트 사용자	본인 계정 이메일 추가 ← 빠뜨리면 로그인 시 차단된다

④ 클라이언트 ID 발급

클라이언트 → 클라이언트 만들기

항목	값
애플리케이션 유형	웹 애플리케이션
이름	<code>n8n</code>
승인된 리디렉션 URI	3-6에서 복사한 n8n Redirect URL

발급된 클라이언트 ID와 보안 비밀번호를 n8n에 붙여넣는다.

🔒 이 두 값은 비밀정보다. 캡처하지 않고, 문서에도 적지 않는다. 팝업을 닫으면 시크릿을 다시 볼 수 없으니 바로 입력한다.

⑤ 구글 로그인

n8n에서 **Sign in with Google** → 계정 선택 → "이 앱은 확인되지 않았습니다" 경고에서 **고급** → **n8n-local (안전하지 않음)**으로 이동 → **권한 3개 모두 체크** → 계속

권한 3개(Drive 메타데이터 / Drive 파일 / Sheets)를 모두 허용해야 목록 조회와 쓰기가 동작한다.

✅ 성공하면 Credential 칸이 **Google Sheets account** 로 채워진다. 별도 완료 메시지는 뜨지 않는다.

🕒 이 구간 소요 시간을 별도로 기록한다. Make는 동일 작업이 1~2분이다.

3-7. Action 1 — Google Sheets (true 경로)

항목	값
Resource	Sheet Within Document
Operation	Append Row
Document	문의접수-자동화 (From list)
Sheet	긴급문의 (From list)
Mapping Column Mode	Map Each Column Manually

Values to Send — 시트 헤더 이름이 그대로 칸 라벨로 표시된다.

칸	입력 방법
수신시각	Expression 모드 → <code>{{ \$now.toISO() }}</code>
ticket_id	INPUT 패널에서 body → ticket_id 드래그
customer	드래그
category	드래그
priority	드래그
message	드래그
source	Fixed 모드 → n8n 타이핑

✅ **Make와의 차이 2가지** ① 매핑이 **헤더 이름 기반**이다 (Make는 A~Z 위치 기반) ② 표현식 아래에 **치환 결과값이 즉시 미리 보기**로 표시된다 → Make의 "무음 실패"가 구조적으로 발생하기 어렵다

📸 **n8n-02c-sheets-mapping.png** — 헤더 라벨과 미리보기 값이 보이는 화면

3-8. Action 2 — Discord 알림 (true 경로)

Sheets 노드 우측 **+** → **HTTP Request**

항목	값
Method	POST
URL	긴급알림 Webhook URL
Send Body	켜기
Body Content Type	JSON
Specify Body	Using JSON

```
{"content": "🔥 긴급 문의 접수\nID: {{ $json.ticket_id }}\n고객: {{ $json.customer }}\n분류: {{ $json.category }}\n내용: {{ $json.message }}"}
```

⚠️ **참조 경로가 바뀐다.** 이 노드의 입력은 Webhook이 아니라 **Sheets 노드의 출력**이다. Sheets가 평면 구조로 내보내므로 `$json.body.ticket_id` 가 아니라 `$json.ticket_id` 다.

3-9. Action 3-4 — false 경로

IF 노드의 **false 출력**에 같은 구성을 만든다. 노드 선택 후 **Ctrl+C** → **Ctrl+V** 로 복제하면 빠르다.

변경할 것	값
Sheets → Sheet	일반문의
HTTP → URL	일반알림 Webhook URL
HTTP → 문구	 일반 문의 접수

💡 이 시점에 "Node was not executed" 안내가 뜰 수 있다. **오류가 아니다.** 현재 흘러보낸 데이터가 true 경로로 갔기 때문에 false 경로 노드가 실행되지 않았다는 뜻이며, 오히려 조건 분기가 정상 작동한다는 증거다.

📸 `n8n-03-workflow-full.png` — 노드 6개와 true/false 라벨이 보이는 캔버스 전체 좌측 하단 **fit to view** (네 모서리 아이콘)를 누르면 전체가 화면에 들어온다.

🕒 **측정 종료** (워크플로우 구축)

3-10. 분기 테스트 6건 🕒 측정 시작

Test URL 방식으로 진행한다. 각 요청 직전에 하단 **Execute workflow** 를 누른다.

⚠ **여섯 번째 함정 — 테스트 모드는 1회용이다** `Execute workflow` 를 누른 후 **단 한 번의 호출만** 받는다. 한 건 보낼 때마다 다시 눌러야 한다. 안 누르고 보내면 `Click the 'Execute workflow' button on the canvas` 오류가 난다.

💡 n8n 브라우저와 PowerShell을 화면 좌우로 나란히 놓고, PowerShell에서는 **위쪽 화살표 키**로 이전 명령을 불러오면 반복이 편하다.

1번 (긴급) — 3-4에서 이미 전송했다면 생략

```
$body='{ "ticket_id": "T-2001", "customer": "고객A", "category": "결제", "priority": "urgent", "message": "결제가 두 번 청구되었습니다"}'; Invoke-RestMethod -Uri "TestURL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

2번 (긴급)

```
$body='{ "ticket_id": "T-2002", "customer": "고객B", "category": "배송", "priority": "urgent", "message": "주문이 취소되지 않습니다"}'; Invoke-RestMethod -Uri "TestURL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

3번 (긴급)

```
$body='{ "ticket_id": "T-2003", "customer": "고객C", "category": "결제", "priority": "urgent", "message": "카드 결제가 실패합니다"}'; Invoke-RestMethod -Uri "TestURL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

4번 (normal)

```
$body='{ "ticket_id": "T-2004", "customer": "고객D", "category": "환불", "priority": "normal", "message": "환불 절차를 문의합니다"}'; Invoke-RestMethod -Uri "TestURL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

5번 (low)

```
$body='{ "ticket_id": "T-2005", "customer": "고객E", "category": "기타", "priority": "low", "message": "영업시간이 궁금합니다"}'; Invoke-RestMethod -Uri "TestURL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

6번 (priority 필드 없음)

```
$body='{ "ticket_id": "T-2006", "customer": "고객F", "category": "기타", "message": "필드 누락 테스트입니다"}'; Invoke-RestMethod -Uri "TestURL" -Method Post -ContentType "application/json; charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

🕒 측정 종료

3-11. 실행 결과 확인 및 캡처

상단 **Executions** 탭 → 좌측 목록에서 개별 실행 클릭 → 우측 캔버스에 해당 실행 경로가 표시된다.

- 실행된 노드: 진한 색 + 초록 체크
- 실행되지 않은 노드: 회색
- 분기 선: 통과한 쪽만 초록색 + `1 item`

파일명	내용	확인 포인트
n8n-01-docker-run.png	구동 로그	버전, 접속 주소, 라이선스 미인증
n8n-01c-license-prompt.png	유료 기능 안내	라이선스 키 필요 항목
n8n-02-webhook-node.png	Trigger 인식	body 중첩 구조
n8n-02b-if-node.png	IF 노드	True/False 탭
n8n-02c-sheets-mapping.png	컬럼 매핑	헤더 라벨 + 미리보기 값
n8n-03-workflow-full.png	전체 구조	노드 6개 + true/false
n8n-04-run-branch-a.png	긴급 경로 실행	true 활성화 / false 회색
n8n-05-run-branch-b.png	일반 경로 실행	위와 정확히 반대
n8n-06-sheets-result.png	시트 결과	두 도구 데이터가 source 로 구분

💡 어느 실행이 몇 번 데이터인지 모르겠으면, 실행을 연 뒤 **Webhook 노드**를 클릭해 **ticket_id** 를 확인한다.

4. 검증 체크리스트

재현이 제대로 되었는지 아래로 확인한다.

요구사항 충족

- ☐ 서로 다른 도구 **2개**로 구현했다
- ☐ 두 도구의 워크플로우 **구조가 동일**하다 (Trigger·분기 조건·Action 구성)
- ☐ **Trigger** 1개 이상 (Webhook)
- ☐ **Action** 2개 이상 (경로당 2개, 총 4개)
- ☐ **조건 분기** 1개 이상 (Make: Router+Filter / n8n: IF)
- ☐ 분기 **양쪽 경로**가 각각 최소 1회 이상 실행된 결과가 있다
- ☐ 무료 플랜 범위 내에서 완수했다

결과 확인

- ☐ **긴급문의** 시트에 **urgent** 건만 들어갔다
- ☐ **일반문의** 시트에 나머지 건만 들어갔다
- ☐ **priority** 필드가 없는 건(T-x006)이 **일반 경로**로 처리되었다 (두 도구 모두)
- ☐ Discord 두 채널에 각각 분리되어 도착했다

☐ 시트·Discord의 변수 값이 비어 있지 않다

보안 (캡처 전 필수)

- ☐ Make Webhook URL 마스킹
- ☐ Discord Webhook URL 노출 없음 (노드 라벨 하단 주의)
- ☐ Google 클라이언트 ID / 시크릿 미기재
- ☐ 계정 이메일·실명 노출 없음 (Discord 서버명, 브라우저 프로필, GCP 콘솔)
- ☐ 테스트 데이터에 실명·실제 연락처 없음

종료 처리

- ☐ Make Webhook 재발급 (기존 URL 폐기)
- ☐ Discord Webhook 재발급 (선택)
- ☐ n8n 컨테이너 정리 (선택)

5. 자주 막히는 지점 요약

재현 중 여기서 시간을 잃기 쉽다. 순서대로 기억해 두면 좋다.

#	도구	증상	원인 / 해결
1	공통	한글이 <code>?????</code> 로 깨짐	<code>charset=utf-8</code> + <code>UTF8.GetBytes</code> 사용
2	공통	PowerShell <code>>></code> 오류	<code>Ctrl+C</code> 후 한 줄로 붙여넣기
3	Make	<code>no scenario listening</code>	<code>Detect new values</code> / <code>Run once</code> 를 먼저 누른다
4	Make	실행은 성공인데 값이 비어 있음	변수를 손으로 입력하지 말고 패널에서 클릭 삽입
5	Make	fallback route를 못 찾음	<code>Not equal to</code> 필터로 우회
6	Make	같은 티켓이 여러 번 기록됨	큐에 쌓였던 요청이 소진된 것. 중복 행 정리
7	n8n	Docker <code>WSL not installed</code>	<code>wsl --install</code> + 재부팅
8	n8n	시트 목록이 비어 있음	Drive API 도 활성화했는지 확인
9	n8n	구글 로그인 차단	OAuth 동의 화면에 테스트 사용자 등록
10	n8n	GCP가 다른 계정으로 열림	URL에 <code>?authuser=N</code> 또는 크롬 프로필 분리
11	n8n	IF 조건이 항상 false	참조 경로에 <code>body.</code> 가 빠졌는지 확인
12	n8n	Webhook 404	UUID 오타 / <code>Execute workflow</code> 미클릭 / 컨테이너 종료

#	도구	증상	원인 / 해결
13	n8n	워크플로우가 아예 안 돌아감	컨테이너가 켜져 있는지 확인 (창을 닫으면 종료됨)

6. 측정 기록 양식

재현하면서 아래 표를 채우면 비교 보고서를 다시 쓸 수 있다.

구간	Make	n8n	비고
초기 진입	분	분	Make: 가입+플랜확인 / n8n: Docker·WSL·컨테이너
워크플로우 구축	분	분	생성 → 첫 성공 실행
└ 이 중 Google 인증	분	분	
분기 테스트 6건	분	분	

막힌 지점 기록 (시간보다 이쪽이 보고서에서 더 중요하다)

도구	막힌 지점	소요	해결 방법

💡 총 시간만 있으면 "Make 54분 / n8n 70분"이라는 숫자는 나오지만 **왜 그런지를 설명하지 못한다**. 어디서 막혔고 어떻게 풀었는지가 비교 보고서의 실제 내용이 된다.

부록 — 참조용 페이로드

```
{
  "ticket_id": "T-1001",
  "customer": "고객A",
  "category": "결제",
  "priority": "urgent",
  "message": "결제가 두 번 청구되었습니다"
}
```

변수 참조 문법 비교

용도	Make	n8n
Webhook 필드 참조	<code>{{ticket_id}}</code> (평면)	<code>{{ \$json.body.ticket_id }}</code> (중첩)
이전 노드 출력 참조	모듈 번호 기반	<code>{{ \$json.필드명 }}</code>
현재 시각	<code>{{now}}</code>	<code>{{ \$now.toISO() }}</code>
함수 인자 구분자	세미콜론 <code>;</code>	쉼표 <code>,</code>